AIPO 2025
Online Round

February 15, 2025

Contributors

The AIPO organisers would like to thank the following people.

For leading the question writing:

- Dr Sabin Tabrica
- Andrew Nash
- Ştefan-Bogdan Marcu
- Yanlin Mi
- Fangda Zou

Time limits are available on the submission system.

1 Persistently Pesky Palindromic Problems

A palindrome is a word that consists only of regular lowercase alphabetic characters (a-z) that is read **exactly** the same forwards and backwards - such as “madam”, “tenet”, “a”, ‘rotavator”, “dad” or “anna”.

Given N strings of characters, for each string you **must** insert a single alphabetic character. If it is possible to turn the string into a palindrome, print this resulting palindrome. Otherwise print the word “NONE”.

It is possible that the original string is a palindrome, but this is not guaranteed. If there are multiple possible solutions, print any of them.

Input/Output

Input

- The first line a single integer N
- N lines follow, each of which contains a string w_i

Output

N lines, each line i containing either the word “NONE” or the resulting palindrome from adding a single character to w_i.

Constraints

- $1 \leq N \leq 100$
- $1 \leq \text{len}(w_i) \leq 15$

Sample Input/Output

Examples

Sample Input

```
4
noon
not
statsy
racecar
```

Sample Output

```
nooon
NONE
ystatsy
raceecar
```

2 Greatly Garbled Guide

You are planning to travel to Galapagos, and your friend who has travelled there before knows the best time of the year to visit and has recommended you a very specific date to travel. Unfortunately, their message was garbled in transit and now you are not sure what their recommendation was.

Given the message that you received from your friend, consisting of numbers and the character "-", find the date that you should visit Galapagos. Dates are given in the format "DD-MM-YYYY". Obviously, you can only plan to travel in the future, so the date must be strictly after today (15-02-2025), take place strictly before 01-01-2028 and it must correspond to a valid date. It is guaranteed that there is one (and only one) date that meets these criteria in the garbled message which will occur one or more times.

Input/Output

Input

- The first line a single integer N, the length of the message in characters
- The second line contains the garbled message

Output

The extracted date, in the format "DD-MM-YYYY"

Constraints

- $10 \leq N \leq 200$

Sample Input/Output

Sample Input 1

```
26
2025-01-01-01-2025-02-2026
```

Sample Output 1

```
25-02-2026
```

Sample Input 2

```
45
1-11-2004--0210230-9214-01-01-2028-12-04-2027
```

Sample Output 2

```
12-04-2027
```

3 Colony Resource Rebalancing

You are the supply officer for the new colony on planet New Eden. Life on New Eden depends on a precise balance of resources—there are n essential supply types (numbered from 1 to n) that keep the colony running. Unfortunately, due to delivery mishaps, your current stock of supplies might not match the colony's minimum needs.

Luckily, you have devised a clever method to reallocate supplies. In a single **redistribution operation**, you can:

- **Choose one supply type** i (with $1 \leq i \leq n$).
- **Increase** the amount of supply type i by 1 unit.
- **Decrease** the amount of every other supply type j (for all $j \neq i$) by 1 unit.

Important: You can only perform this operation if every supply type you'd have to reduce has at least 1 unit available (no resource may become negative).

The colony's survival depends on meeting or exceeding the required minimum for each supply type. Specifically, the colony needs at least b_i units of supply type i for all $1 \leq i \leq n$.

Your task is to determine whether it is possible to achieve the required amounts by performing the **redistribution operation** any number of times (possibly zero).

Input/Output

Input

The first line contains a single integer t ($1 \leq t \leq 1000$) — the number of test cases. The description of the test cases follows.

Each test case begins with a line containing an integer n ($2 \leq n \leq 2 \cdot 10^5$) — the number of supply types.

The next line contains n integers $a_1, a_2, \ldots, a_n$ ($0 \leq a_i \leq 10^9$) — the initial number of units you have for each supply type.

The third line contains n integers $b_1, b_2, \ldots, b_n$ ($0 \leq b_i \leq 10^9$) — the required number of units for each supply type to sustain the colony.

It is guaranteed that the total sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, print a single line containing either "YES" if it is possible to meet or exceed the required quantities using a series of redistribution operations, or "NO" otherwise.

Examples

Sample Input

```
3
4
0 10 9 1
1 9 8 0
3
2 2 3
3 3 2
2
1 10
5 5
```

Sample Output 1

```
YES
NO
YES
```

Explanation of Sample Input/Output

Test Case 1 You start with $[0, 10, 9, 1]$ and need $[1, 9, 8, 0]$. Perform a redistribution operation on supply type 1: - Supply type 1 increases: $0 + 1 = 1$ - Supply types 2, 3, and 4 decrease: $10 - 1 = 9$, $9 - 1 = 8$, $1 - 1 = 0$ The supplies now exactly match the requirements.

Test Case 2 No matter how you perform the operations, you cannot reallocate your supplies to meet the targets.

Test Case 3 By performing the redistribution operation on supply type 1 twice, you can boost it to the required level while still having enough of the other supply (or even surplus) to meet their minimum needs.

4 Careful Crafty Card Counting

You are given a series of N cards, each of which have a (not necessarily unique) number c_i.

In the game of "card counting", your task is to select a number K. Once you have selected K, you must pick up all cards $j_1, j_2, \ldots$ for which number c_j is divisible by K.

For example, for cards $5, 2, 10, 13, 50$ and $K = 2$ you must pick up the second, third and fifth cards.

If you choose a value of K for which you do not pick up any two consecutive cards, and do not leave any two consecutive cards on the table - you win. Otherwise, you lose.

In the example above for $K = 2$, you would lose as you picked up two consecutive cards (with numbers 2 and 10). If you had chosen $K = 5$ you would win - as you would take the first, third and fifth cards - no consecutive pairs are taken or left on the table.

Given C games of "card counting" on different series of cards, determine for each what the winning value of K is. If there is no possible winning value, print "UNWINNABLE". If there are multiple valid winning values, print any of them.

Input/Output

Input

- The first line a single integer C, the number of games that follow. Each of the following $2 \times C$ lines contain:
  - A line N_i, the number of cards in game i
  - Followed by a line containing the numbers c_j printed on the cards for game i

Output

C lines each containing either single integer K, or the string "UNWINNABLE"

Constraints

- $1 \leq C \leq 10$
- $1 \leq N \leq 150$
- $1 \leq c_j \leq 10^{16}$

Sample Input/Output

Sample Input 1

```
2
3
83 33 9
3
62 49 82
```

Sample Output 1

```
33
2
```

Sample Input 2

```
2
5
84 89 8 75 72
5
15 63 23 34 92
```

Sample Output 2

```
4
UNWINNABLE
```

5 Terrific Train Travels Through Towns

Given a set of N towns numbered $0, 1, \ldots, N-1$, and E train tracks between pairs of towns, your task is to calculate the number of possible types of train that can be used on some route between certain pairs of important towns.

Three are K different types of train, and each individual train track between towns i and j can only be used by one particular type of train. These trains can travel on either direction down these tracks.

It is possible that there are multiple tracks between two towns, but it is guaranteed that in this case no two of tracks between a pair of towns will be for the same type of train.

You are given C questions about pairs of towns t_i and t_j. Find the number of possible trains that can be used on some route from t_i to t_j, where the train starts at t_i and only travels down tracks of its specific type, until it reaches t_j.

Input/Output

Input

- The first line will contain space-separated integers N and E
- E lines follow, each of which contain space-separated integers t_i, t_j k_i - which means that there is a track of type k_i between towns t_i and t_j
- The next line contains the single integer C
- C lines follow, each of which contain space-separated integers t_i, t_j - meaning that you need to compute the number of possible trains between these two towns.

Output

C lines, each containing a single integer - the total number of trains (possibly 0) that can be used on routes between the given pairs of towns.

Constraints

- $2 \leq N \leq 100$
- $1 \leq E \leq 750$
- $0 \leq t_i, t_j < N$
- $1 \leq C \leq 100$
- $0 \leq k_i < E$

Sample Input/Output

Sample Input 1

```
2 3
0 1 0
0 1 2
0 1 2
1
1 0
```

Sample Output 1

```
2
```

Sample Input 2

```
5 8
0 3 0
2 3 3
0 2 0
0 3 0
1 3 2
2 3 1
1 3 2
0 2 2
2
1 3
0 1
```

Sample Output 2

```
1
0
```

6 Enchanted Runes

In the ancient kingdom of **QWERTY**, the High Mage has discovered a corridor in the Grand Archive lined with mystical runes. There are n runes arranged in a row, numbered from 1 to n. Each rune carries a magical power level denoted by a_i.

To cast a powerful spell, the High Mage must select a contiguous sequence of these runes. The spell's potency is determined by how dramatically the power levels vary within the chosen sequence—but with a twist. In particular, if the mage selects runes from position l to r, the potency is calculated as

$$\text{potency} = (\max\{a_l, a_{l+1}, \ldots, a_r\} - \min\{a_l, a_{l+1}, \ldots, a_r\}) - (r - l).$$

This formula captures the idea that while a larger spread in power levels can yield a mightier spell, including too many runes (i.e. a longer sequence) may actually dilute the effect.

However, the magical energies are ever-changing. Over time, q events occur where the power of a single rune fluctuates. In each event, the rune at position p changes its power level to a new value x.

Your task is to help the High Mage by determining the maximum spell potency achievable—first for the initial arrangement of runes, and then after each magical fluctuation.

Input/Output

Input

The input for each test consists of several test cases. The first line contains a single integer t ($1 \leq t \leq 1000$) — the number of test cases.

For each test case:

- The first line contains two integers n and q ($1 \leq n \leq 2 \cdot 10^5, 0 \leq q \leq 2 \cdot 10^4$) — the number of runes and the number of power fluctuations.
- The next line contains n integers $a_1, a_2, \ldots, a_n$ ($1 \leq a_i \leq 10^9$) — the initial power levels of the runes.
- Each of the following q lines contains two integers p and x ($1 \leq p \leq n, 1 \leq x \leq 10^9$) indicating that the power level of the rune at position p changes to x.

Output

For each test case, output $q + 1$ numbers in one line, representing the maximum spell potency that can be achieved by selecting an optimal contiguous segment of runes:

- First, output the answer for the initial configuration.
- Then, output the answer after each power fluctuation.

Constraints

- For 10% of the tests, The sum of n over all test cases does not exceed 2000, $q = 0$.
- For another 30% of the tests, The sum of n over all test cases does not exceed $2 \cdot 10^5$, $q = 0$.
- For the rest 60% of the tests, The sum of n does not exceed $2 \cdot 10^5$, the sum of q over all test cases does not exceed $2 \cdot 10^4$.

Examples

Sample Input

```
4
2 2
2 8
1 8
2 1
5 3
2 3 4 5 6
3 8
1 5
5 3
8 6
8 6 1 3 2 1 5 4
5 3
1 9
3 3
8 13
7 1
3 1
5 0
1 4 6 8 4
```

Sample Output 1

```
5 0 6
0 4 4 4
5 5 6 4 10 11 11
4
```

Explanation of Sample Input/Output

Consider the first test case:

- Initially, if the High Mage selects both runes, the spell's potency is calculated as $(8-2)-(2-1) = 5$.
- After the first fluctuation, both runes have a power level of 8, so even the best option (selecting a single rune) yields a potency of $8 - 8 - 0 = 0$.
- After the second fluctuation, the runes' power levels become $8, 1$, and selecting both yields a potency of $8 - 1 - (2 - 1) = 6$.